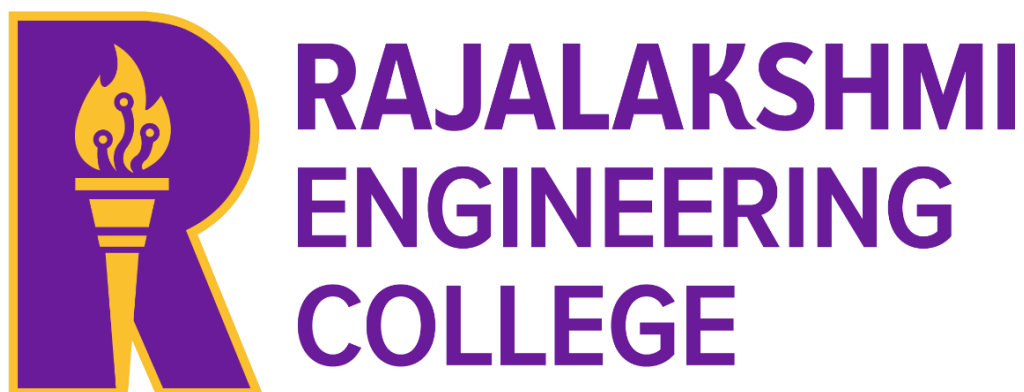


RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution)

RAJALAKSHMI NAGAR, THANDALAM- 602 105



AD23B31 - Image Processing and Computer Vision

LABORATORY RECORD NOTEBOOK

NAME: Raghavan S

YEAR/SEMESTER: III - VI

BRANCH: Computer Science Engineering

REGISTER NO: 230701520

COLLEGE ROLL NO: 2116230701520

ACADEMIC YEAR: 2025-2026



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution)

RAJALAKSHMI NAGAR, THANDALAM- 602 105

BONAFIDE CERTIFICATE

NAME: Raghavan S

ACADEMIC YEAR: 2025 - 26

REGISTER NO: 230701520

BRANCH/SECTION:

SEMESTER: VI

Certified that this is a Bonafide record of work done by the above student in the **AD23B31 - Image Processing and Computer Vision** during the year 2025 - 2026

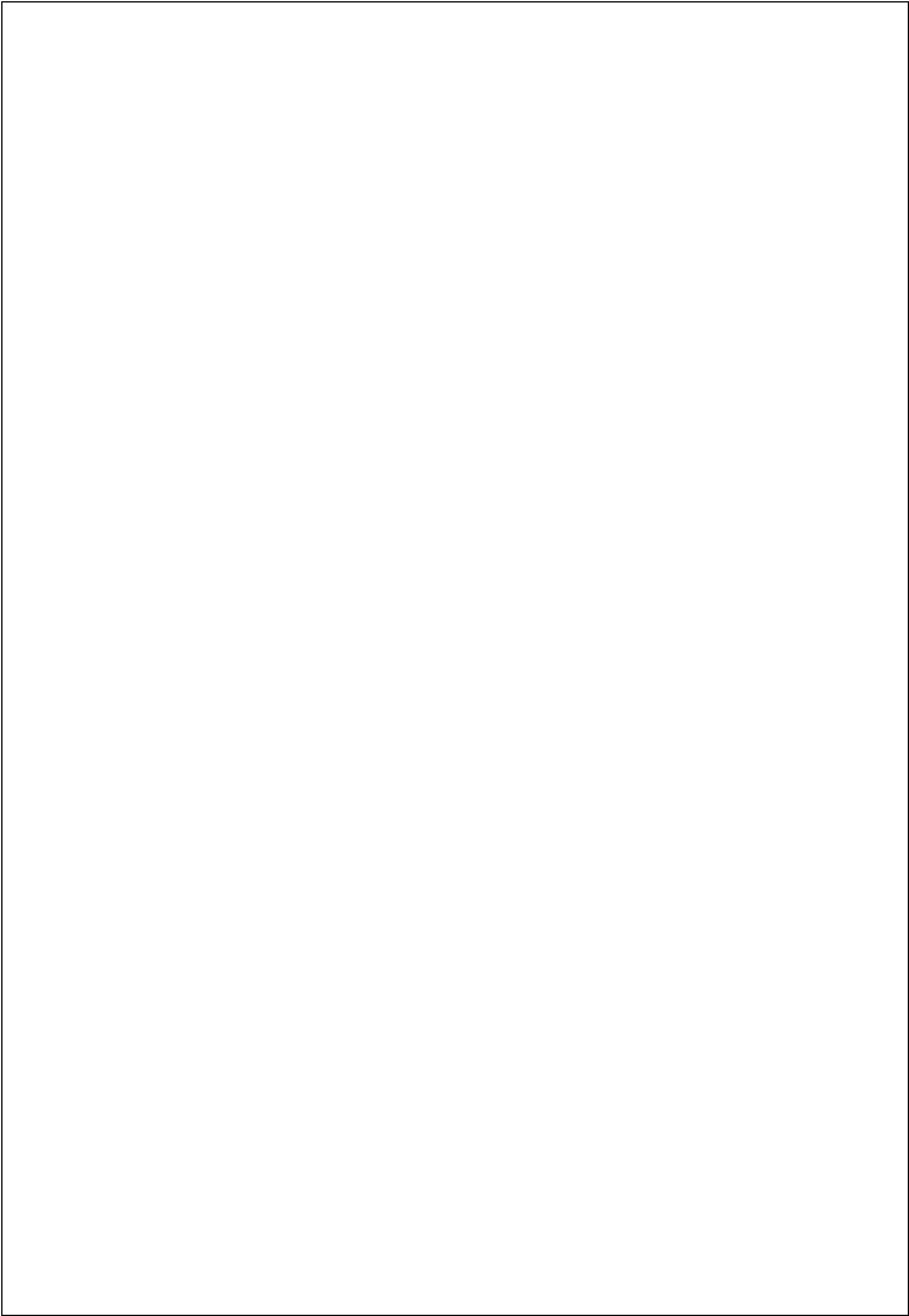
Signature of Faculty In-charge

Submitted for the Practical Examination Held on:

Internal Examiner

External Examiner

EX.NO	DATE	NAME OF THE EXPERIMENT	PAGE NO	STAFF SIGN
1	10/1/2026	Installation of Open CV and to perform the basic image handling processing operation on the image		
2	24/1/2026	2D Image Transformation		
3	31/1/2026	Implement Edge Detection, Line Detection and Corner Detection		
4	7/2/2026	Demonstrate Camera Calibration using python		
5	14/2/2026	Implement Image Histogram and Histogram Equalization		
6	21/2/2026	Develop a python program for Skin Color Detection		
7	7/3/2026	Create a python program for Warping and Estimation		
8	14/3/2026	Develop a python program for Motion Tracking		
9	4/4/2026	Design a program for Object Detection using YOLO		
10	11/4/2026	Develop a python program for Stereo Vision and Depth Estimation		
11	25/4/2026	Demonstrate Augmented Reality using Feature Matching		
12	25/4/2026	Mini Project		



1. Installation of Open CV and to perform the basic image handling processing operation on the image

AIM :

To install the OpenCV library and perform basic image handling operations such as reading, displaying, converting, and saving images using Python.

PROGRAM:

```
!pip install numpy
!pip install matplotlib
!pip install opencv-python
!pip install ipython

# import the libraries
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Image
import cv2
%matplotlib inline
import cv2
import matplotlib.pyplot as plt

cb_img = cv2.imread("checkboard.png", cv2.IMREAD_GRAYSCALE)
print(cb_img)

plt.imshow(cb_img, cmap='gray')
plt.axis('off')
plt.show()

import cv2
import matplotlib.pyplot as plt

cb_img = cv2.imread("checkboard.png", cv2.IMREAD_GRAYSCALE)

print("Shape of the image:", cb_img.shape)
print("Data type of the image:", cb_img.dtype)
print("Size of the image:", cb_img.size)
import cv2
import matplotlib.pyplot as plt

flower = cv2.imread("flower.jpg", cv2.IMREAD_COLOR)
b, g, r = cv2.split(flower)

plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.imshow(r, cmap="gray")
```

```
plt.title("Red Channel")
plt.axis("off")
```

```
plt.subplot(1,3,2)
plt.imshow(g, cmap="gray")
plt.title("Green Channel")
plt.axis("off")
```

```
plt.subplot(1,3,3)
plt.imshow(b, cmap="gray")
plt.title("Blue Channel")
plt.axis("off")
```

```
import cv2
import matplotlib.pyplot as plt
```

```
flower = cv2.imread("flower.jpg", cv2.IMREAD_COLOR)
```

```
hsv = cv2.cvtColor(flower, cv2.COLOR_BGR2HSV)
h, s, v = cv2.split(hsv)
```

```
plt.figure(figsize=[20, 5])
```

```
plt.subplot(1, 4, 1)
plt.imshow(h, cmap="gray")
plt.title("H Channel")
plt.axis("off")
```

```
plt.subplot(1, 4, 2)
plt.imshow(s, cmap="gray")
plt.title("S Channel")
plt.axis("off")
```

```
plt.subplot(1, 4, 3)
plt.imshow(v, cmap="gray")
plt.title("V Channel")
plt.axis("off")
```

```
plt.subplot(1, 4, 4)
plt.imshow(cv2.cvtColor(flower, cv2.COLOR_BGR2RGB))
plt.title("Original")
plt.axis("off")
```

```
plt.show()
```

```

import cv2
import matplotlib.pyplot as plt

flower = cv2.imread("flower.jpg", cv2.IMREAD_COLOR)
flower1 = cv2.cvtColor(flower, cv2.COLOR_BGR2GRAY)

plt.imshow(flower1, cmap="gray")
plt.title("BGR to grayscale")
plt.axis("off")
plt.show()

import cv2
import matplotlib.pyplot as plt

flower = cv2.imread("flower.jpg", cv2.IMREAD_COLOR)
reverse = flower[:, :, ::-1]

plt.imshow(reverse)
plt.axis("off")
plt.show()
# Convert from BGR to RGB
import cv2
import matplotlib.pyplot as plt

flower = cv2.imread("flower.jpg", cv2.IMREAD_COLOR)
flower1 = cv2.cvtColor(flower, cv2.COLOR_BGR2RGB)

plt.imshow(flower1)
plt.title("BGR to RGB")
plt.axis("off")
plt.show()

```

BGR to grayscale



1:



`(np.float64(-0.5), np.float64(188.5), np.float64(266.5), np.float64(-0.5))`

Red Channel



Green Channel



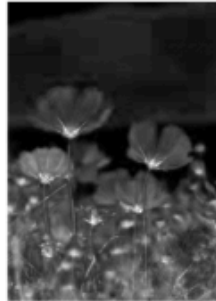
Blue Channel



H Channel



S Channel



V Channel



Original



BGR to grayscale



2. 2D Image Transformation

AIM :

To perform 2D image transformations using OpenCV, including operations such as translation, rotation, scaling, and flipping on digital images using Python.

PROGRAM:

```
# Read and Display Image
import cv2
import numpy as np
import matplotlib.pyplot as plt

flower = cv2.imread("flower1.jpg")
flower1 = cv2.cvtColor(flower, cv2.COLOR_BGR2RGB)

plt.imshow(flower1)
plt.title("Original Image")
plt.axis("off")
plt.show()

# Cropping
import cv2
import matplotlib.pyplot as plt

flower = cv2.imread("flower1.jpg")
# Slicing: [start_row:end_row, start_col:end_col]
cropped = flower[50:300, 100:400]
flower1 = cv2.cvtColor(cropped, cv2.COLOR_BGR2RGB)

plt.imshow(flower1)
plt.title("Cropped Image")
plt.axis("off")
plt.show()

# Rotation
import cv2
import matplotlib.pyplot as plt

flower = cv2.imread("flower1.jpg")
rows, cols = flower.shape[:2]

center = (cols // 2, rows // 2)
# Parameters: center, angle, scale
M = cv2.getRotationMatrix2D(center, 45, 1.0)
rotated = cv2.warpAffine(flower, M, (cols, rows))

flower1 = cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB)

plt.imshow(flower1)
plt.title("Rotated Image")
```

```

plt.axis("off")
plt.show()
# Perspective Transformation
import cv2
import numpy as np
import matplotlib.pyplot as plt

flower = cv2.imread("flower1.jpg")
rows, cols = flower.shape[:2]

pts1 = np.float32([[50, 50], [cols-50, 50], [50, rows-50], [cols-50, rows-50]])
pts2 = np.float32([[10, 100], [cols-100, 50], [100, rows-10], [cols-50, rows-50]])

M = cv2.getPerspectiveTransform(pts1, pts2)
perspective = cv2.warpPerspective(flower, M, (cols, rows))

flower1 = cv2.cvtColor(perspective, cv2.COLOR_BGR2RGB)

plt.imshow(flower1)
plt.title("Perspective Transformation")
plt.axis("off")
plt.show()
# Resizing and Scaling
# Method 1: Absolute dimensions
resized = cv2.resize(flower, (300, 300))

# Method 2: Scaling by factors (e.g., half size)
scaled = cv2.resize(flower, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_LINEAR)

flower1 = cv2.cvtColor(scaled, cv2.COLOR_BGR2RGB)
plt.imshow(flower1)
plt.title("Scaled Image (0.5x)")
plt.show()
# Translation
rows, cols = flower.shape[:2]
# Shift 100 pixels right, 50 pixels down
M = np.float32([[1, 0, 100], [0, 1, 50]])
translated = cv2.warpAffine(flower, M, (cols, rows))

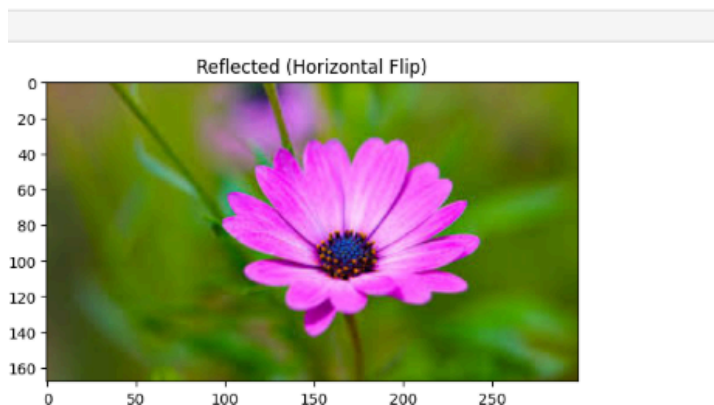
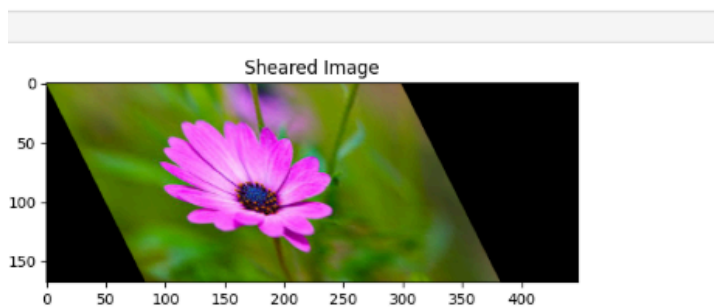
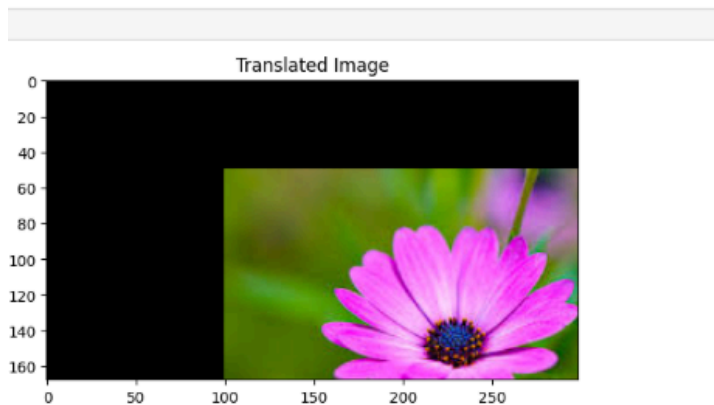
flower1 = cv2.cvtColor(translated, cv2.COLOR_BGR2RGB)
plt.imshow(flower1)
plt.title("Translated Image")
plt.show()

# Shearing (Horizontal)
M = np.float32([[1, 0.5, 0], [0, 1, 0]])
sheared = cv2.warpAffine(flower, M, (int(cols*1.5), rows))

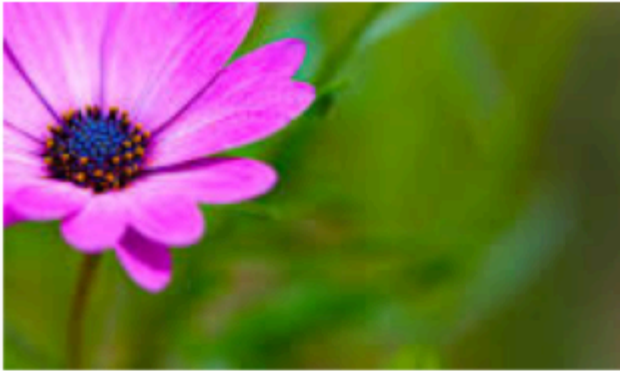
```

```
flower1 = cv2.cvtColor(sheared, cv2.COLOR_BGR2RGB)
plt.imshow(flower1)
plt.title("Sheared Image")
plt.show()
# Reflection
# 0: Vertical flip, 1: Horizontal flip, -1: Both
reflected = cv2.flip(flower, 1)
```

```
flower1 = cv2.cvtColor(reflected, cv2.COLOR_BGR2RGB)
plt.imshow(flower1)
plt.title("Reflected (Horizontal Flip)")
plt.show()
```



Cropped Image



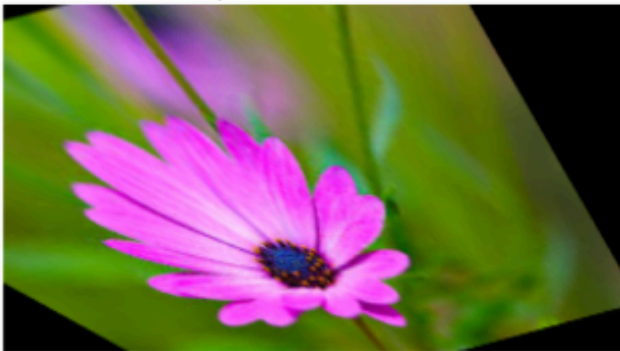
:

Rotated Image



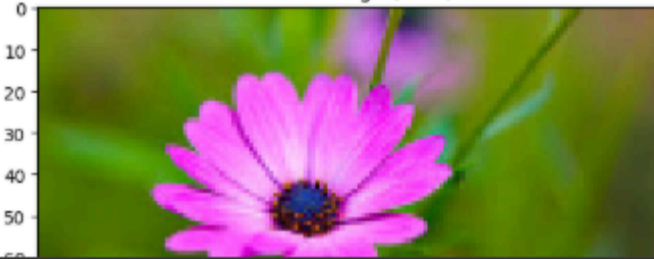
:

Perspective Transformation



:

Scaled Image (0.5x)



3.Implement Edge Detection, Line Detection and Corner Detection

AIM :

To implement edge detection, line detection, and corner detection techniques on images using OpenCV in Python.

PROGRAM :

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("flower1.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

Ix = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
Iy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

edges = cv2.magnitude(Ix, Iy)
# Fixed: changed 'p.max' to 'np.max'
edges = np.uint8(255 * edges / np.max(edges))

plt.figure(figsize=(10,5))

plt.subplot(121)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.title("Original Image")

plt.subplot(122)
plt.imshow(edges, cmap='gray')
plt.axis("off")
plt.title("Sobel Edges")

plt.show()
#Canny Edge Detection
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("flower.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

edges = cv2.Canny(gray, 100, 200)

plt.figure(figsize=(10,5))

plt.subplot(121)
```

```

plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.title("Original Image")

plt.subplot(122)
plt.imshow(edges, cmap='gray')
plt.axis("off")
plt.title("Canny Edges")

plt.show()
#Laplacian Edge Detection
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("flower.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

edges = cv2.Laplacian(gray, cv2.CV_64F)
edges = np.uint8(255 * np.abs(edges) / np.max(np.abs(edges)))

plt.figure(figsize=(10,5))

plt.subplot(121)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.title("Original Image")

plt.subplot(122)
plt.imshow(edges, cmap='gray')
plt.axis("off")
plt.title("Laplacian Edges")

plt.show()
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("flower.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Defining Prewitt Kernels
kernelx = np.array([[1,1,1],[0,0,0],[-1,-1,-1]])
kernely = np.array([[-1,0,1],[-1,0,1],[-1,0,1]])

img_prewittx = cv2.filter2D(gray, -1, kernelx)
img_prewitty = cv2.filter2D(gray, -1, kernely)
edges = img_prewittx + img_prewitty

```

```

plt.figure(figsize=(10,5))

plt.subplot(121)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.title("Original Image")

plt.subplot(122)
plt.imshow(edges, cmap='gray')
plt.axis("off")
plt.title("Prewitt Edges")

plt.show()
# Robert's Cross Edge Detection
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("flower1.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Defining Roberts Kernels
kernelx = np.array([[1, 0], [0, -1]])
kernely = np.array([[0, 1], [-1, 0]])

img_robertsx = cv2.filter2D(gray, -1, kernelx)
img_robertsy = cv2.filter2D(gray, -1, kernely)
edges = img_robertsx + img_robertsy

plt.figure(figsize=(10, 5))

plt.subplot(121)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.title("Original Image")

plt.subplot(122)
plt.imshow(edges, cmap='gray')
plt.axis("off")
plt.title("Robert's Edges")

plt.show()
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image

```

```

img = cv2.imread("flower1.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Gamma value
# Gamma > 1: Image gets darker
# Gamma < 1: Image gets brighter
gamma = 2.2

# Power Law Transformation:  $s = c * r^{\gamma}$ 
# We normalize the image to [0, 1] before applying the power
c = 255 / (np.max(gray)**gamma)
# Fixed: changed 'p.max' to 'np.max' and ensured float calculation
gamma_corrected = np.array(c * (gray.astype(np.float64)**gamma), dtype=np.uint8)

plt.figure(figsize=(10,5))

plt.subplot(121)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.title("Original Image")

plt.subplot(122)
plt.imshow(gamma_corrected, cmap='gray')
plt.axis("off")
plt.title(f"Gamma Corrected (gamma={gamma})")

plt.show()

```

Original Image



Sobel Edges



Original Image



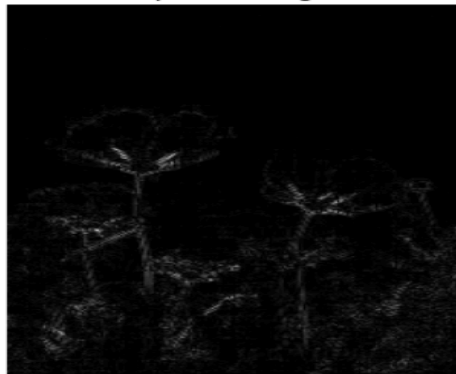
Canny Edges



Original Image



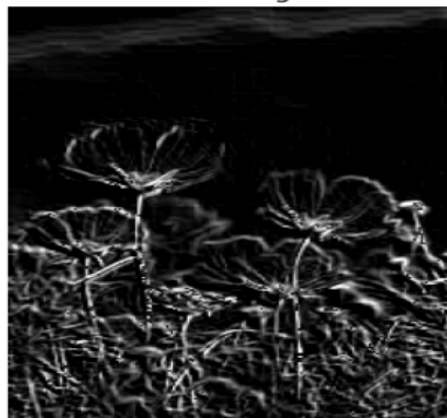
Laplacian Edges



Original Image



Prewitt Edges



Original Image



Robert's Edges



4 . Demonstrate Camera Calibration using python

AIM :

To demonstrate camera calibration using Python and OpenCV in order to estimate camera parameters and correct image distortion.

PROGRAM :

```
import cv2
import numpy as np
import glob
import os

# --- 1. SET THESE TO MATCH YOUR ACTUAL BOARD ---
# If your board has 8x10 squares, use 8 and 10.
# If you are using the OpenCV sample images, use 9 and 6.
num_squares_x = 8
num_squares_y = 10

corners_x = num_squares_x - 1
corners_y = num_squares_y - 1

checker_size = 0.020
path = r"D:\images" # Added 'r' here to fix the path error!

criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)

objp = np.zeros((corners_x * corners_y, 3), np.float32)
objp[:, :2] = np.mgrid[0:corners_y, 0:corners_x].T.reshape(-1, 2)
objp = objp * checker_size

objpoints = []
imgpoints = []

# Get list of images
images = glob.glob(os.path.join(path, '*.jpg'))

print(f"Found {len(images)} images in {path}")

if len(images) == 0:
    print("Error: No images found! Check your folder path and file extensions.")
else:
    for fname in images:
        img = cv2.imread(fname)
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

        # Find the chess board corners
        ret, corners = cv2.findChessboardCorners(gray, (corners_y, corners_x), None)
```

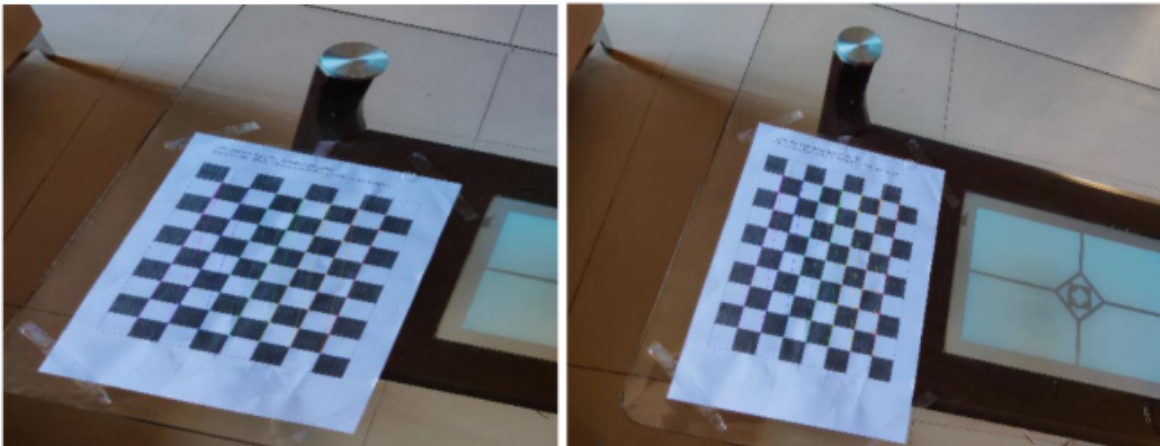
```

if ret:
    print(f"Corners found in: {fname}")
    objpoints.append(objp)
    corners2 = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1), criteria)
    imgpoints.append(corners2)
else:
    print(f"Corners NOT found in: {fname} - Check if num_squares is correct.")

# Only run calibration if we found corners in at least one image
if len(objpoints) > 0:
    ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(objpoints, imgpoints, gray.shape[::-1], None,
None)

    print("\n--- Extrinsic Parameters ---")
    for i, (rvec, tvec) in enumerate(zip(rvecs, tvecs)):
        R, _ = cv2.Rodrigues(rvec)
        print(f"\nImage {i+1}")
        print(f"Rotation Matrix:\n{R}")
        print(f"Translation Vector:\n{tvec}")
    else:
        print("\nCalibration failed: No corners were detected in any of the images.")

```



5. Implement Image Histogram and Histogram Equalization

AIM :

To implement image histogram analysis and histogram equalization using OpenCV in Python to enhance image contrast.

PROGRAM :

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
img = cv2.imread("flower.jpg",cv2.IMREAD_GRAYSCALE)
values,count = np.unique(img,return_counts=True)
pixels = img.size
pdf = count/pixels
cdf = np.cumsum(pdf)
trans = np.floor(255*cdf).astype(np.uint8)
equalized = np.zeros_like(img)
for i, v in enumerate(values):
    equalized[img==v] = trans[i]

opencv = cv2.equalizeHist(img)
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(img,cmap="gray")
plt.title("Original")
plt.axis("Off")

plt.subplot(1,3,2)
plt.imshow(equalized,cmap="gray")
plt.title("2")
plt.axis("Off")

plt.subplot(1,3,3)
plt.imshow(opencv,cmap="gray")
plt.title("3")
plt.axis("Off")

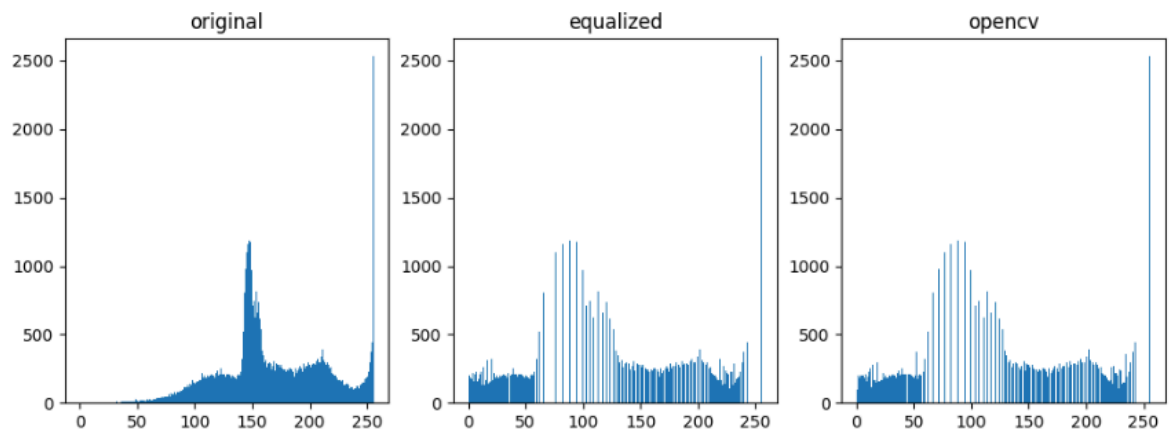
plt.show()
plt.figure(figsize=(12,4))
plt.subplot(1,3,1)
plt.hist(img.ravel(),bins=256,range=[0,256])
plt.title("original")

plt.subplot(1,3,2)
plt.hist(equalized.ravel(),bins=256,range=[0,256])
plt.title("equalized")
```

```
plt.subplot(1,3,3)
plt.hist(opencv.ravel(),bins=256,range=[0,256])
plt.title("opencv")
```



Text(0.5, 1.0, 'opencv')



6. Develop a python program for Skin Color Detection

AIM :

To develop a Python program for detecting human skin color regions in images using OpenCV based on color space segmentation techniques.

PROGRAM :

```
import os
import numpy as np
import matplotlib.pyplot as plt
import imageio.v2 as imageio

%matplotlib inline

face_dir = "images/SkinColor/FacePhoto"
mask_dir = "images/SkinColor/MaskPhoto"

images = []
masks = []
names = []

image_files = os.listdir(face_dir)
mask_files = os.listdir(mask_dir)

for img_file in image_files:
    name = os.path.splitext(img_file)[0]
    for mask_file in mask_files:
        mask_name = os.path.splitext(mask_file)[0]
        if name == mask_name:
            img_path = os.path.join(face_dir, img_file)
            mask_path = os.path.join(mask_dir, mask_file)

            image = imageio.imread(img_path)
            mask = imageio.imread(mask_path)

            images.append(image)
            masks.append(mask)
            names.append(img_file)
            break

print("Number of Face Images Loaded:", len(images))

for i in range(len(images)):
    image = images[i]
    mask = masks[i]
    name = names[i]
```

```

mask_rgb = mask[:, :, :3]

skin_pixels = np.all(mask_rgb == [255, 255, 255], axis=-1)
skin_image = np.zeros_like(image)

skin_image[skin_pixels] = image[skin_pixels]

total_pixels = skin_pixels.size
skin_count = np.sum(skin_pixels)
skin_percentage = (skin_count / total_pixels) * 100

print("Image:", name)
print("Skin Pixels:", skin_count)
print("Skin Percentage: %.2f%%" % skin_percentage)
print("-----")

plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(mask)
plt.title("Mask")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(image)
plt.imshow(mask, alpha=0.4)
plt.title("Overlay")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(skin_image)
plt.title("Extracted Skin")
plt.axis("off")

plt.suptitle(name)
plt.show()

if i == 2:
    break

```

Original Image



Mask



Overlay



Extracted Skin



7. Create a python program for Warping and Estimation

AIM :

To create a Python program for image warping and geometric transformation estimation using OpenCV to map one image plane onto another.

PROGRAM :

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Load image
img = cv2.imread("images/warping.jpg")
h, w = img.shape[:2]
print("Image size:", w, "x", h)

# Display original image
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()

# Define source points for perspective transform
src_pts = np.float32([
    [40, 60],
    [250, 50],
    [260, 350],
    [30, 360]
])

# Define destination points and dimensions
width = 300
height = 400
dst_pts = np.float32([
    [0, 0],
    [width, 0],
    [width, height],
    [0, height]
])

# Apply perspective transform
H = cv2.getPerspectiveTransform(src_pts, dst_pts)
warped = cv2.warpPerspective(img, H, (width, height))

# Display warped image
warped_rgb = cv2.cvtColor(warped, cv2.COLOR_BGR2RGB)
```

```
plt.imshow(warped_rgb)
plt.title("Warped Image")
plt.axis("off")
plt.show()
```

Original Image



Warped Image



8. Develop a python program for Motion Tracking

AIM :

To develop a Python program for motion tracking by detecting and following moving objects in video sequences using OpenCV.

PROGRAM :

```
import cv2
import matplotlib.pyplot as plt
from IPython.display import display, clear_output

cap = cv2.VideoCapture("Motion tracking/car.mp4")

ret, frame1 = cap.read()
if not ret:
    print("Error: Cannot read video")
    exit()

prev_gray = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
prev_gray = cv2.GaussianBlur(prev_gray, (11, 11), 0)

print("Car Motion Tracking Started...")

while True:
    ret, frame2 = cap.read()
    if not ret:
        break

    gray = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)
    gray = cv2.GaussianBlur(gray, (11, 11), 0)

    diff = cv2.absdiff(prev_gray, gray)

    _, thresh = cv2.threshold(diff, 20, 255, cv2.THRESH_BINARY)

    thresh = cv2.dilate(thresh, None, iterations=2)
    contours, _ = cv2.findContours(
        thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    )

    for c in contours:
        if cv2.contourArea(c) > 1000:
            x, y, w, h = cv2.boundingRect(c)
            cv2.rectangle(frame2, (x, y), (x + w, y + h), (0, 255, 0), 2)

    frame_rgb = cv2.cvtColor(frame2, cv2.COLOR_BGR2RGB)
    clear_output(wait=True)
```

```
plt.imshow(frame_rgb)
plt.title("Car Motion Tracking")
plt.axis('off')
display(plt.gcf())
plt.close()

prev_gray = gray

cap.release()
print("Tracking Stopped")
```

Car Motion Tracking



9. Design a program for Object Detection using YOLO

AIM :

To design a program for object detection using the YOLO (You Only Look Once) algorithm with OpenCV in Python.

PROGRAM :

```
import cv2
import numpy as np

import os

base = "yolo"
cfg = os.path.join(base, "yolov3-tiny.cfg")
weights = os.path.join(base, "yolov3-tiny.weights")
classes = open("yolo/coco.names").read().splitlines()
img = cv2.imread("yolo/person.jpg")

net = cv2.dnn.readNet(weights, cfg)

if img is None:
    print("Image not found")
    exit()

height, width = img.shape[:2]

blob = cv2.dnn.blobFromImage(img, 1/255, (416, 416), swapRB=True)
net.setInput(blob)

layer_names = net.getLayerNames()
output_layers = [layer_names[i - 1] for i in net.getUnconnectedOutLayers()]

outputs = net.forward(output_layers)

boxes = []
confidences = []
class_ids = []

for out in outputs:
    for det in out:
        scores = det[5:]
        class_id = np.argmax(scores)
        confidence = scores[class_id]

        if confidence > 0.5:
            cx, cy, w, h = (det[0:4] * [width, height, width, height]).astype("int")
```

```
x = int(cx - w / 2)
y = int(cy - h / 2)
```

```
boxes.append([x, y, w, h])
confidences.append(float(confidence))
class_ids.append(class_id)
```

```
indexes = cv2.dnn.NMSBoxes(boxes, confidences, 0.5, 0.4)
```

```
for i in indexes:
```

```
    i = i[0] if isinstance(i, (list, tuple, np.ndarray)) else i
```

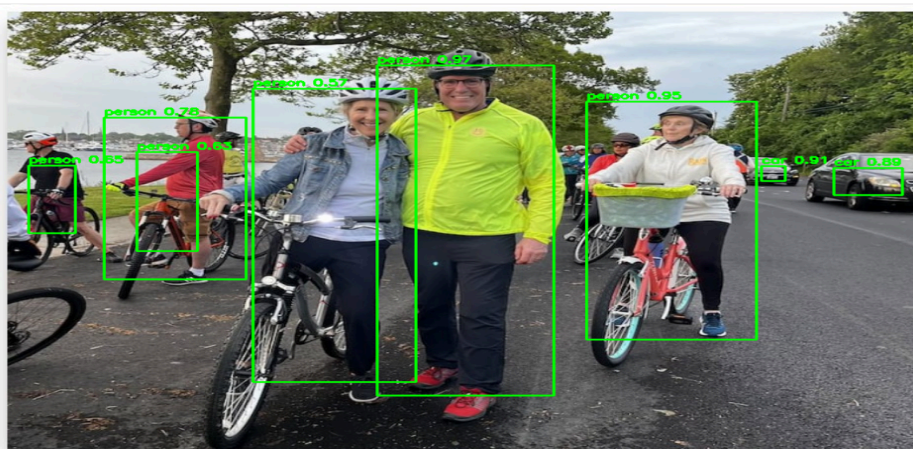
```
    x, y, w, h = boxes[i]
    label = classes[class_ids[i]]
    conf = confidences[i]
```

```
    cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)
    cv2.putText(
        img,
        f"{label} {conf:.2f}",
        (x, y - 5),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.5,
        (0, 255, 0),
        2
    )
```

```
cv2.imshow("YOLO Detection", img)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```



10. Develop a python program for Stereo Vision and Depth Estimation

AIM :

To develop a Python program for stereo vision and depth estimation using OpenCV to compute depth information from two images

PROGRAM :

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

imgL = cv2.imread(r"D:\ipcv\stereo estimation\left.png")

imgR = cv2.imread(r"D:\ipcv\stereo estimation\right.png")

if imgL is None or imgR is None:

    print("Error: Images not found in 'stereo estimation/' folder")

    exit()

imgL = cv2.cvtColor(imgL, cv2.COLOR_BGR2GRAY)

imgR = cv2.cvtColor(imgR, cv2.COLOR_BGR2GRAY)

imgR = cv2.resize(imgR, (imgL.shape[1], imgL.shape[0]))

stereo = cv2.StereoSGBM_create(

    minDisparity=0,

    numDisparities=16 * 5,

    blockSize=5,

    P1=8 * 3 * 25,

    P2=32 * 3 * 25,

    uniquenessRatio=10,

    speckleWindowSize=100,

    speckleRange=32

)
```

```

disparity = stereo.compute(imgL, imgR)

disp = cv2.normalize(disparity, None, 0, 255, cv2.NORM_MINMAX)

disp = np.uint8(disp)

depth = np.zeros_like(disparity, dtype=np.float32)

depth[disparity > 0] = 1000 / (disparity[disparity > 0] + 1)

plt.figure(figsize=(10, 4))

plt.subplot(1, 3, 1)

plt.title("Left Image")

plt.imshow(imgL, cmap='gray')

plt.subplot(1, 3, 2)

plt.title("Right Image")

plt.imshow(imgR, cmap='gray')

plt.subplot(1, 3, 3)

plt.title("Depth Map")

plt.imshow(disp, cmap='jet')

plt.show()

valid_depth = depth[depth > 0]

print("\n===== DEPTH SUMMARY =====")

print(f"Avg Depth : {np.mean(valid_depth):.2f}")

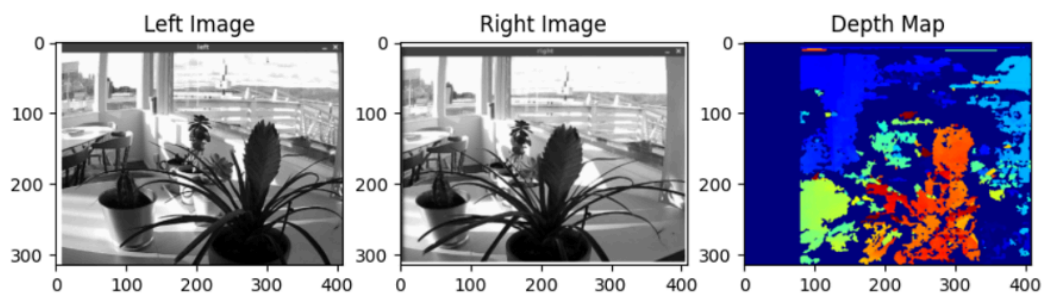
print(f"Min Depth : {np.min(valid_depth):.2f}")

print(f"Max Depth : {np.max(valid_depth):.2f}")

print("=====\\n")

```

Ex-10



```
===== DEPTH SUMMARY =====  
Avg Depth : 4.02  
Min Depth : 0.79  
Max Depth : 100.00  
=====
```

11. Demonstrate Augmented Reality using Feature Matching

AIM :

To demonstrate augmented reality using feature matching techniques with OpenCV in Python by overlaying virtual objects onto real-world scenes.

PROGRAM :

```
import cv2
import numpy as np

ref_img = cv2.imread("D:/ipcv/AR/reference.jpg", 0)
scene_img = cv2.imread("D:/ipcv/AR/scene.jpg")
overlay_img = cv2.imread("D:/ipcv/AR/overlay.png")

if ref_img is None or scene_img is None or overlay_img is None:
    raise Exception("Error: Check image paths")

overlay_img = cv2.resize(overlay_img, (ref_img.shape[1], ref_img.shape[0]))

orb = cv2.ORB_create(nfeatures=1000)

kp1, des1 = orb.detectAndCompute(ref_img, None)
kp2, des2 = orb.detectAndCompute(
    cv2.cvtColor(scene_img, cv2.COLOR_BGR2GRAY), None
)

bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

matches = bf.match(des1, des2)
matches = sorted(matches, key=lambda x: x.distance)

good_matches = matches[:50]

if len(good_matches) < 10:
    raise Exception("Not enough matches found")

src_pts = np.float32([kp1[m.queryIdx].pt for m in good_matches]).reshape(-1, 1, 2)
dst_pts = np.float32([kp2[m.trainIdx].pt for m in good_matches]).reshape(-1, 1, 2)

H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)

h, w = ref_img.shape

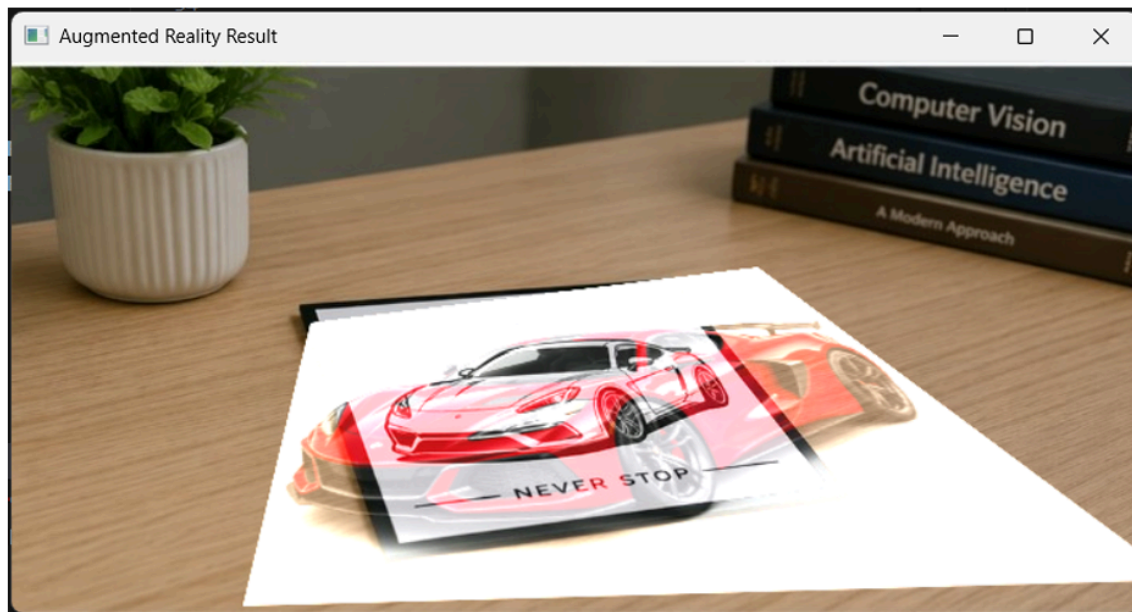
overlay_warp = cv2.warpPerspective(
    overlay_img, H, (scene_img.shape[1], scene_img.shape[0])
)
```

```
result = cv2.addWeighted(scene_img, 1, overlay_warp, 1, 0)
```

```
cv2.imshow("Augmented Reality Result", result)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```



Program :

```
#include <iostream>
```

```
#include <string>
```

```
#include <spdlog/spdlog.h>
```

```
#include "core/config.h"
```

```
#include "core/logger.h"
```

```
#include "core/pipeline.h"
```

```
int main(int argc, char* argv[]) {
```

```
    std::string config_path = "config/pipeline.yaml";
```

```
    for (int i = 1; i < argc; i++) {
```

```
        std::string arg = argv[i];
```

```
        if ((arg == "--config" || arg == "-c") && i + 1 < argc) {
```

```
            config_path = argv[++i];
```

```
        } else if (arg == "--help" || arg == "-h") {
```

```
            std::cout << "Usage: drone_tracker [--config <path>]\n"
```

```
                << "\nKeyboard controls:\n"
```

```
                << "  Q/ESC  Quit\n"
```

```
                << "  T    Cycle target selection mode\n"
```

```
                << "  1-9  Select track by ID\n"
```

```
                << "  R    Toggle recording\n"
```

```
                << "  D    Toggle debug overlay\n";
```

```
            return 0;
```

```
        }
```

```
    }
```

```
    try {
```

```
        auto config = drone_tracker::Config::load(config_path);
```

```
        drone_tracker::init_logger(config.logging);
```

```
        spdlog::info("Drone Tracker v0.1.0");
```

```
        spdlog::info("Config: {}", config_path);
```

```
        drone_tracker::Pipeline pipeline(config);
```

```
        pipeline.start();
```

```
} catch (const std::exception& e) {  
    spdlog::error("Fatal: {}", e.what());  
    std::cerr << "Error: " << e.what() << std::endl;  
    return 1;  
}  
  
return 0;  
}
```

FULL PROJECT LINK : <https://github.com/Raghavan2005/drone-tracker>

